Nexus — 3–4 YOE Scenario-Based Q&A

1. What is Nexus Repository?
Answer:
Nexus Repository is an artifact repository used to store and manage application packages such as Maven, npm, Docker images, and other build artifacts.
It allows teams to publish, download, and control access to artifacts from a central location.
Simple ga gurthupettuko:
Build → Nexus → Store → Download
Commands:
docker login nexus.example.com
docker push nexus.example.com/myapp:v1.0
docker pull nexus.example.com/myapp:v1.0

---

2. What is the difference between Hosted, Proxy, and Group repositories?
Answer:
A **Hosted** repository stores artifacts maintained by our organization.
A **Proxy** repository caches artifacts from an external repository such as Maven Central.
A **Group** repository combines multiple hosted and proxy repositories behind one URL.
Simple ga gurthupettuko:
Hosted → Own artifacts | Proxy → External cache | Group → Combined

---

3. How do you upload an artifact to Nexus?
Answer:
First, I configure the Nexus repository URL and credentials in the build tool.
Then the CI pipeline builds the artifact and publishes it to the appropriate hosted repository.
Simple ga gurthupettuko:
Build → Authenticate → Upload → Verify
Commands:
mvn clean package
mvn deploy

---

4. Docker image push to Nexus is failing. What will you check?
Answer:
First, I check the Nexus Docker repository URL and whether the Docker repository is running.
Then I verify Docker authentication, repository permissions, image tag, and network connectivity.
I also check Nexus logs if the push is still failing.
Simple ga gurthupettuko:
URL → Repo → Login → Permission → Tag → Network
Commands:
docker login nexus.example.com
docker tag myapp:v1 nexus.example.com/myapp:v1
docker push nexus.example.com/myapp:v1

---

5. Nexus is running but artifacts cannot be downloaded. What will you check?
Answer:
I check repository availability, artifact coordinates/version, user permissions, and repository configuration.
Then I check Nexus logs and network connectivity to identify the failure.
Simple ga gurthupettuko:
Repo → Artifact → Permission → Network → Logs
Commands:
curl -I https://nexus.example.com
docker pull nexus.example.com/myapp:v1

---

6. Nexus disk usage is very high. What will you check?
Answer:
First, I identify which repository is consuming most of the storage.
Then I check old artifacts, Docker image layers, retention policies, and cleanup policies.
I remove only approved unused artifacts and configure proper cleanup policies.
Simple ga gurthupettuko:
Disk → Repository → Old Artifacts → Cleanup Policy
Commands:
df -h
du -sh *

7. How do you manage Nexus artifacts across DEV, QA, and PROD?
Answer:
I maintain controlled repositories and versioned artifacts instead of rebuilding the same application for each environment.
CI builds the artifact once, stores it in Nexus, and promotes the same version through DEV, QA, and PROD.
Simple ga gurthupettuko:
Build Once → Nexus → DEV → QA → PROD

---

8. How do you integrate Nexus with Jenkins?
Answer:
Jenkins builds the application and publishes the artifact to Nexus using configured credentials.
During deployment, Jenkins downloads the required artifact version from Nexus and deploys it to the target environment.
Simple ga gurthupettuko:
Jenkins Build → Nexus Upload → Nexus Download → Deploy

---

9. Nexus authentication is failing. What will you check?
Answer:
First, I verify the username, password/token, and repository permissions.
Then I check whether the account is active and whether the repository requires specific authentication settings.
Simple ga gurthupettuko:
Credentials → Account → Permission → Repository
Commands:
curl -u username:password https://nexus.example.com/
docker login nexus.example.com

---

10. Maven build is failing because it cannot download dependencies from Nexus. What will you check?
Answer:
I check the Nexus repository URL configured in Maven settings.
Then I verify repository availability, credentials, proxy/group configuration, and whether the required dependency exists.
Simple ga gurthupettuko:
Maven URL → Nexus → Credentials → Group Repo → Dependency
Commands:
cat ~/.m2/settings.xml
mvn dependency:tree
mvn clean install

---

11. Nexus proxy repository is not downloading artifacts from the internet. What will you check?
Answer:
First, I check whether the external repository URL is correct and reachable.
Then I verify Nexus outbound network access, DNS, proxy configuration, and repository status.
Simple ga gurthupettuko:
External URL → Network → DNS/Proxy → Nexus Repo
Commands:
curl -I https://repo1.maven.org
nslookup repo1.maven.org

---

12. Same artifact version is being overwritten. How will you handle it?
Answer:
I prefer immutable versioning so that once an artifact is released, it is not overwritten.
For example, I use unique application versions or build numbers for every release.
Simple ga gurthupettuko:
One Version → One Artifact → Don't Overwrite
Example:
myapp:1.0.1
myapp:1.0.2
myapp:1.0.3

---

13. How do you secure Nexus?
Answer:
I use role-based access, strong authentication, HTTPS, and separate permissions for read and write operations.
I also restrict administrative access and regularly review users and permissions.
Simple ga gurthupettuko:
HTTPS → RBAC → Least Privilege → Admin Restriction

---

14. Nexus is down. What will you check?
Answer:
First, I check the Nexus service/container status and server resources.
Then I check disk space, memory, logs, database/storage health, and network access.
After identifying the issue, I restore the service and verify artifact upload/download.
Simple ga gurthupettuko:
Service → Resources → Disk → Logs → Storage → Network
Commands:

systemctl status nexus
df -h
free -m
tail -f /path/to/nexus/log/nexus.log

---

15. How do you handle Nexus backup and recovery?
Answer:
I take regular backups of Nexus configuration and repository data according to the organization's backup strategy.
I also test restoration periodically to ensure the backup can actually recover the repository.
Simple ga gurthupettuko:
Backup → Store Safely → Test Restore → Recovery

---

16. CI pipeline successfully built the artifact but Nexus upload failed. What will you check?
Answer:
First, I verify Nexus availability and repository URL.
Then I check CI credentials, repository permissions, artifact version, and network connectivity.
After fixing the issue, I retry the upload.
Simple ga gurthupettuko:
Build OK → Nexus → Credentials → Permission → Network → Upload

---

17. Docker image exists in Nexus but Kubernetes cannot pull it. What will you check?
Answer:
First, I verify the image repository and tag configured in Kubernetes.
Then I check the Nexus Docker registry availability and Kubernetes image pull credentials.
I also check Pod events for ImagePullBackOff or authentication errors.
Simple ga gurthupettuko:
Image/Tag → Nexus → imagePullSecret → Events
Commands:
kubectl describe pod <pod_name> -n prod
kubectl get secret -n prod
docker pull nexus.example.com/myapp:v1

---

18. How do you troubleshoot Nexus in a Production environment?
Answer:
First, I check the impact and Nexus availability.
Then I check service status, disk/memory, repository health, authentication, network connectivity, and logs.
I identify the root cause and restore artifact access with minimal impact.
Simple ga gurthupettuko:
Impact → Service → Resources → Repo → Auth/Network → Logs → Fix

---

19. How do you promote artifacts from DEV to PROD?
Answer:
I build the artifact once and store it in Nexus with a unique version.
After DEV and QA validation, I promote the same tested artifact to PROD instead of rebuilding it.
Simple ga gurthupettuko:
Build Once → Nexus → Test → Promote Same Artifact → PROD

---

20. Real-time CI/CD → Jenkins → Nexus → Kubernetes scenario. Explain.
Answer:
Developer pushes code to Git, which triggers Jenkins.
Jenkins builds and tests the application, creates a versioned artifact or Docker image, and pushes it to Nexus.
Then Jenkins deploys that exact version to Kubernetes and verifies the rollout.
If the deployment fails, we rollback to the previous known-good version.
Simple ga gurthupettuko:
Git → Jenkins → Build → Nexus → Kubernetes → Verify → Rollback
Commands:
mvn clean package
docker build -t nexus.example.com/myapp:v1.2.0 .
docker push nexus.example.com/myapp:v1.2.0
kubectl set image deployment/myapp myapp=nexus.example.com/myapp:v1.2.0
kubectl rollout status deployment/myapp